DALI ServUI

Dokumentation

Version: 1.0.0

Datum: 2026-05-07

Autor: c42u

Co-Autor: ClaudeCode

Lizenz: GPLv3

Table of Contents

1. Glossary
2. Disclaimer
3. Background and Project Idea
4. Software Used
5. Architecture
6. Supported Hardware
7. Installation
8. Configuration
9. Operation
10. REST API
11. Server-Sent Events (SSE)
12. Multi-Dashboard
13. Commissioning
14. Bus Log and Sniffer
15. Security
16. Backup and Restore
17. Troubleshooting
18. What's new in v1.0.0
19. Appendix A – Checklist

1. Glossary

Term	Description
DALI	Digital Addressable Lighting Interface – IEC 62386, the standard for digital lighting control
DALI bus	Two-wire communication line (DALI+ / DALI-), polarity-independent, max. 64 devices
Forward Frame	16-bit command from the master to the devices (1 start bit + 8 bit address + 8 bit command)
Backward Frame	8-bit reply from a device to the master
Short address	Unique device address 0–63 on the DALI bus
Commissioning	Process of automatic address assignment to DALI devices

Term	Description
DAPC	Direct Arc Power Control – brightness control (0–254)
Manchester encoding	Line code in which every bit is represented by a level transition
HID	Human Interface Device – USB device class for input devices
FTDI	Future Technology Devices International – manufacturer of USB-to-serial converters
mikroBUS	Standardised socket from MikroElektronika for Click boards
Dryrun	Test mode without connected hardware

2. Disclaimer

Safety notice and liability disclaimer

Use of DALI ServUI is **at your own risk**. The author assumes no liability for personal injury, property damage or financial loss arising from the use of this software. This software was developed with assistance from AI technology (Claude, Anthropic).

Installation and commissioning

DALI ServUI controls electrical lighting installations. The installation, wiring and commissioning of the DALI bus system – in particular all work on the mains-voltage side of the ballasts – must be carried out exclusively by a **qualified, licensed electrician**. The applicable national regulations (e.g. IEC 60364, BS 7671, NEC) must be observed. Improper configuration or faulty wiring can lead to malfunction of the lighting, equipment damage or, in the worst case, personal injury.

License

GNU General Public License v3.0 (GPLv3). The full license is available at https://www.gnu.org/licenses/gpl-3.0.html.

3. Background and Project Idea

Background

The existing project **DALIPi** implemented a CLI-based DALI lighting control via the Hasseb USB DALI Master. Control happened from the command line and via a TCP bridge with a line-based ASCII protocol. In parallel, the open-source project **daliserver** (github.com/onitake/daliserver) exists, a C-based TCP daemon for the Tridonic DALI USB adapter.

Both approaches had limitations: no web interface, no containerisation, limited hardware support.

Project idea

DALI ServUI combines the strengths of both projects into a modern, containerised solution with a web interface:

- **Multi-hardware support:** Hasseb USB, MikroE DALI Click (GPIO + USB), test mode
- **Web UI:** Responsive interface with dashboard, devices, groups and discovery views
- **REST API:** Full API for integration with other systems
- **Docker + native:** Runs as a container or as a direct Linux installation
- **Bilingual:** German and English

The architecture follows the daliserver approach (queue-based command multiplexing) and is implemented in Python with Flask + Jinja2.

4. Software Used

Software	Version	Source	Purpose
Python	3.12	python.org	Programming language
Flask	3.x	pypi.org/project/Flask	Web framework
Gunicorn	22.x	pypi.org/project/gunicorn	WSGI server
hidapi	0.14.x	pypi.org/project/hidapi	USB HID access (Hasseb)
gpiod	2.x	pypi.org/project/gpiod	GPIO access (MikroE/RPi)
pyftdi	0.55.x	pypi.org/project/pyftdi	FTDI USB access (MikroE)
Docker	27.x	docker.com	Containerisation
Jinja2	3.x	(bundled with Flask)	Template engine

5. Architecture

Layer model

```
┌──────────────────────────────────────────────┐
│  Web browser (dashboard, control, help)      │
└──────────────────────────────────────────────┘
                │
                │ HTTP / REST API
┌───────────────▼──────────────────────────────┐
│  Flask + Jinja2 (main.py)                    │
│  - Web routes + templates                    │
│  - REST API v1                               │
│  - Translations (DE/EN)                      │
└──────────────────────────────────────────────┘
                │
                │
┌───────────────▼──────────────────────────────┐
│  DALI Service (dali_service.py)              │
│  - Queue-based command multiplexing          │
│  - Worker thread (sequential processing)     │
│  - High-level DALI commands                  │
│  - Bus scan + commissioning                  │
└──────────────────────────────────────────────┘
                │
                │ Plugin interface
┌───────────────▼──────────────────────────────┐
│  Driver registry (drivers/)                  │
│  ├── hasseb.py      (USB-HID)                │
│  ├── mikroe_gpio.py (Raspberry Pi GPIO)       │
│  ├── mikroe_ftdi.py (FTDI FT2232H USB)        │
│  └── dryrun.py      (test mode)              │
└──────────────────────────────────────────────┘
                │
                │
┌───────────────▼──────────────────────────────┐
│  DALI hardware (bus, luminaires, ballasts)   │
└──────────────────────────────────────────────┘
```

Plugin driver architecture

All drivers inherit from the abstract base class `DaliDriver` and implement:

- `open()` – open the hardware connection
- `close()` – close the hardware connection
- `send_frame()` – send a DALI forward frame, optionally wait for a reply
- `get_info()` – driver information for the web UI

The registry (`drivers/registry.py`) registers drivers via lazy import – missing dependencies (e.g. `gpiod` on a PC without GPIO) are tolerated. The driver is then marked as “not available”.

File overview

File	Function
`Code/app/main.py`	Flask main application, routes, API
`Code/app/dali_service.py`	DALI service with queue and high-level commands
`Code/app/config.py`	Configuration (environment variables)
`Code/app/translations.py`	Translations DE/EN
`Code/app/drivers/base.py`	Abstract driver base class
`Code/app/drivers/hasseb.py`	Hasseb USB-HID driver
`Code/app/drivers/mikroe_gpio.py`	MikroE GPIO driver (RPi)
`Code/app/drivers/mikroe_ftdi.py`	MikroE FTDI driver (USB)
`Code/app/drivers/dryrun.py`	Simulated test mode
`Code/app/drivers/registry.py`	Driver registry
`Code/app/templates/`	Jinja2 templates (6 pages)
`Code/app/static/`	CSS + JavaScript

6. Supported Hardware

Hasseb USB DALI Master

- **Manufacturer:** Hasseb (hasseb.fi)
- **Interface:** USB-HID (plug & play, no drivers needed)
- **USB:** Vendor 0x04CC, Product 0x0802
- **Bus power supply:** Integrated, 250mA max.
- **DALI version:** DALI 2 compatible
- **Protocol:** 10-byte HID packets with sequence numbers

MikroElektronika DALI Click *(coming soon)*

> ***Note:*** *The integration of the MikroE boards (both GPIO and FTDI/USB) is marked coming soon in v0.8.0. The driver stubs are present in the code, but the hardware test with real ballasts is still ongoing (opto-coupler levels are borderline at 3.3V). In the settings dialog, these drivers are therefore tagged with a "coming soon" stamp and cannot be activated through the UI.*

- **Manufacturer:** MikroElektronika (mikroe.com)
- **Interface:** GPIO (TX=RST, RX=INT on mikroBUS)
- **Bus power supply:** External, required
- **TX logic:** Not inverted (TX HIGH = bus inactive)
- **Connection:** Pi Click Shield or Click USB Adapter

MikroElektronika DALI 2 Click *(coming soon)*

- **Manufacturer:** MikroElektronika (mikroe.com)
- **Interface:** GPIO (TX=RST, RX=INT on mikroBUS)
- **Bus power supply:** External, required
- **TX logic:** Inverted (TX HIGH = bus active) → enable the "TX inverted" option!
- **DALI version:** DALI 2 compatible
- **Connection:** Pi Click Shield or Click USB Adapter

Connection options for MikroE boards

Method	Hardware	Python library
Raspberry Pi direct	Pi Click Shield (connectors soldered)	gpiod
USB on any PC	Click USB Adapter (FT2232H)	pyftdi

7. Installation

Option A: Docker

```
# Clone repository
git clone https://github.com/c42u/dali-servui.git
cd dali-servui

# Start container
cd Docker/
docker compose up -d
```

The container is reachable at `http://<IP>:5000`.

For USB hardware, the container needs access to the USB device. In `docker-compose.yml`, `privileged: true` is preset. Alternatively, a specific device can be mapped:

```
devices:
  - /dev/hidraw0:/dev/hidraw0
```

Option B: Native Linux installation

```
# Run installation script
sudo ./Code/install.sh install
```

The script performs the following steps:

1. Install system dependencies (python3, libhidapi, libusb)
2. Create user `dali` and group `dali`
3. Set up the udev rule for USB access
4. Create a Python venv and install dependencies
5. Set up the systemd service `dali-servui.service`

```
# Start service
sudo systemctl start dali-servui

# Check status
sudo systemctl status dali-servui

# Show logs
sudo journalctl -u dali-servui -f

# Disable service
sudo systemctl stop dali-servui
```

Uninstallation

```
sudo ./Code/install.sh uninstall
```

Data in `/var/lib/dali-servui` and the user `dali` are preserved.

8. Configuration

Environment variables

Variable	Description	Default
`DALI_DRIVER`	Driver ID: hasseb, mikroe_gpio, mikroe_ftdi, dryrun	(stored config)
`DALI_HOST`	Bind address of the web server	0.0.0.0
`DALI_PORT`	Port of the web server	5000
`DALI_LANG`	Default language (de/en)	de
`DALI_API_TOKEN`	API authentication token (empty = disabled)	
`DALI_LOG_LEVEL`	Log level (DEBUG, INFO, WARNING, ERROR)	INFO
`DALI_LOG_FILE`	Path to log file (empty = stdout only)	
`DALI_DATA_DIR`	Data directory for configuration and backup	Code/data
`DALI_SECRET_KEY`	Flask session key	empty → auto-generated in `data/secret_key` (0600)
`DALI_CORS_ORIGINS`	Comma-separated CORS origins; `*` for all	empty (no CORS header)

Driver configuration via web UI

The driver configuration can be edited directly in the settings of the web interface. The configuration is stored as `driver_config.json` in the data directory and loaded automatically on the next start.

udev rule (Hasseb)

The installer automatically sets up a udev rule:

```
KERNEL=="hidraw*", ATTRS{idVendor}=="04cc", ATTRS{idProduct}=="0802", MODE="0660",
GROUP="dali"
```

9. Operation

Dashboard

The dashboard shows connection status, firmware version, device count and queue size. In the lower area you control all luminaires simultaneously via broadcast. The device cards show the current status of each individual device.

Devices page

Tabular overview of all detected DALI devices with address, brightness (slider), device type, group membership and reachability status.

Groups page

Shows all DALI groups (0–15) with their members. Each group can be switched on/off or dimmed together.

Device discovery

- **Bus scan:** Probes addresses 0–63 (~10–20 seconds)
- **Commissioning (all):** Resets all addresses and assigns new ones
- **Commissioning (new only):** Addresses only devices without a short address

Settings

Driver selection with configuration fields. The change takes effect immediately and is stored persistently. Language switch between German and English.

Help

Built-in help page with full instructions on all features, hardware configuration, API documentation and troubleshooting.

Buy me a coffee

Right next to “Help” in the navbar, the link points to buymeacoffee.com/c42u – a voluntary contribution towards further development. Opens in a new tab and has no effect on functionality.

10. REST API

Base URL: `http://<host>:5000/api/v1/`

If an API token is configured: send the header `X-API-Token: <token>`.

Status

Method	Path	Description
GET	`/status`	Service status (running, connected, driver, firmware, devices)
GET	`/devices`	All detected devices
GET	`/drivers`	Available drivers
GET	`/drivers/config`	Current driver configuration

Device control

Method	Path	Body	Description
POST	`/devices/{addr}/on`	–	Switch device on
POST	`/devices/{addr}/off`	–	Switch device off
POST	`/devices/{addr}/level`	`{"level": 0..254}`	Set brightness

Broadcast

Method	Path	Body	Description
POST	`/broadcast/on`	–	Switch all on
POST	`/broadcast/off`	–	Switch all off

Method	Path	Body	Description
POST	`/broadcast/level`	`{"level": 0..254}`	All to brightness

Groups

Method	Path	Body	Description
POST	`/groups/{g}/on`	–	Switch group on
POST	`/groups/{g}/off`	–	Switch group off
POST	`/groups/{g}/level`	`{"level": 0..254}`	Group brightness

Discovery & drivers

Method	Path	Body	Description
POST	`/scan`	–	Start bus scan
POST	`/commission`	`{"broadcast": true}`	Commissioning
POST	`/raw`	`{"address":0,"command":0,"expect_reply":false}`	Raw DALI command
POST	`/drivers/switch`	`{"driver_id":"hasseb"}`	Switch driver
GET	`/drivers/commission-status`	–	Commissioning status + log snapshot
GET	`/drivers/commission-stream`	–	Live SSE stream during commissioning
POST	`/drivers/stop` / `/start`	–	Stop / start the driver (release USB)

Method	Path	Body	Description
POST	`/sniff`	`{"duration": 10}`	Bus sniffer (passive listen, max. 60s)
POST	`/reset-addresses`	–	Clear all short addresses
POST	`/factory-reset`	–	Full DALI factory reset

Labels and group management

Method	Path	Body	Description
GET	`/labels`	–	All device and group names
POST	`/labels/device/{addr}`	`{"name": "..."}`	Set device name (max. 100 chars)
POST	`/labels/group/{g}`	`{"name": "..."}`	Set group name (max. 100 chars)
GET	`/groups`	–	All groups with names, members, level
POST	`/devices/{addr}/groups`	`{"group": 0..15}`	Add to group
DELETE	`/devices/{addr}/groups`	`{"group": 0..15}`	Remove from group

DT8 Tunable White & RGB

Method	Path	Body	Description
POST	`/devices/{addr}/colour-temp`	`{"kelvin":3000}` or `{"mirek":333}`	Colour temperature

Method	Path	Body	Description
POST	`/devices/{addr}/rgb`	`{"r":0,"g":0,"b":0}`	RGB colour (DT8)
POST	`/broadcast/colour-temp`	as above	Broadcast tunable white
POST	`/broadcast/rgb`	as above	Broadcast RGB
GET	`/features`	–	Current feature flags
POST	`/features`	`{"dt6":true,"dt8_tc":true,"dt8_rgb":false}`	Toggle features

Multi-Dashboard

Method	Path	Body	Description
GET	`/dashboards`	–	All dashboards
GET	`/dashboards/{id}`	–	Single dashboard
POST	`/dashboards`	`{"name":"...","items":[...]}`	Create new dashboard
PUT	`/dashboards/{id}`	–	Update dashboard
DELETE	`/dashboards/{id}`	–	Delete dashboard (default protected)

Bus log and real-time

Method	Path	Body	Description
GET	`/buslog`	`?limit=100&since=...`	Fetch bus frames

Method	Path	Body	Description
DELETE	`/buslog`	–	Clear bus log
POST	`/buslog/toggle`	`{"enabled": true}`	Bus logging on/off
GET	`/events`	–	SSE stream of all state changes

11. Server-Sent Events (SSE)

For real-time updates without polling, the SSE endpoint `GET /api/v1/events` is available. The stream emits JSON events on every state change:

Event type	Fields	Description
`level`	`address`, `level`	Brightness of a device changed
`on` / `off`	`address`	Device switched on/off
`group_level`	`group`, `level`	Group brightness
`group_on` / `group_off`	`group`	Group switched on/off
`scan_complete`	`count`, `devices`	Bus scan finished
`colour_temp`	`address`, `mirek`	DT8 tunable white
`rgb`	`address`, `r`, `g`, `b`	DT8 RGB

Keepalive comments every 30 seconds prevent proxy timeouts. Multiple parallel clients are allowed (thread-safe event bus).

Example (browser):

```
const es = new EventSource('/api/v1/events');
es.onmessage = (e) => console.log(JSON.parse(e.data));
```

12. Multi-Dashboard

Multiple dashboards can be configured per room or use case. Each dashboard is a curated selection of devices and groups with optional status cards and broadcast controls.

Item types

- `all_devices` – all detected devices

- `all_groups` – all groups
- `device` with `address` (0..63) – a single device
- `group` with `id` (0..15) – a single group

Items are validated server-side; invalid addresses or group IDs are rejected.

Persistence

The configuration is stored in `dashboards.json` in the data directory. The `default` dashboard is protected and can neither be renamed nor deleted.

13. Commissioning

Commissioning assigns unique short addresses to all DALI devices on the bus. For Hasseb, DALI ServUI uses the python-dali subprocess procedure, which has been tested stably with more than 20 ballasts.

Sequence

1. The driver is stopped automatically (USB released)
2. All short addresses are cleared (`Initialise` + `DTR0(0xFF)` + `SetShortAddress(Broadcast)`)
3. The commissioning sequence assigns new addresses
4. A bus scan validates the result
5. The driver is restarted, the UI shows the devices immediately

Protection against double execution

- Only one commissioning at a time (`threading.Lock`)
- HTTP 409 if a second start is attempted
- Live output via SSE (`/drivers/commission-stream`)
- Output is buffered in the backend across page changes (`/drivers/commission-status`) – reload shows the previous run

CLI

`Code/dali_commission.py` runs the same procedure standalone (`--reset-only` , `--scan-only` , `--no-reset` as options).

14. Bus Log and Sniffer

The bus log records all TX/RX frames with timestamp and a human-readable description in a ring buffer (500 entries). For pure read-only mode without own master traffic, the Hasseb sniffer mode is available.

Aspect	Details
Frame descriptions	DAPC, DTR0/DTR1, group commands, commissioning frames, DT6/DT8
Activation	On by default, can be disabled via `POST /buslog/toggle`
Sniffer duration	0.5 s up to max. 60 s, clamped by the API
Display	`/buslog` page with auto-refresh, pause, clear

Direct frame entry is possible via `POST /api/v1/raw` (`address` and `command` each 0..255).

15. Security

Authentication and CORS

Aspect	Default	Recommendation
API token	empty (auth off)	set for LAN operation; all clients send `X-API-Token`
CORS origins	empty (no header)	precise origins (e.g. Home Assistant URL) instead of `*`
Bind address	`0.0.0.0`	with reverse proxy set to `127.0.0.1`, otherwise firewall in front of port 5000
TLS	not built-in	reverse proxy (Caddy, Traefik, nginx) in front

If neither an API token is set nor the port is shielded, any web page that a user opens in the same browser can send commands to the DALI luminaires via CORS or simply via CSRF-capable `POST` requests. At least one of the two protections should be active.

Frontend authentication (since v0.9.0)

When `DALI_API_TOKEN` is set, the backend writes the token into a hidden `<meta name="x-api-token">` of the rendered page. The bundled JavaScript reads it on load and sends it as `X-API-Token` header on every API request. SSE connections (`/api/v1/events` , `/api/v1/drivers/commission-stream`) authenticate via a `?token=...` query parameter, because `EventSource` cannot send custom headers.

> ***Threat model note:*** *The token ends up in the DOM and is therefore visible to every browser user who can open the UI. This is intentional. API token = "machine or UI authentication". If you want to restrict the UI itself from browser access, put a reverse proxy with basic auth or OIDC in front – the token still applies to external API clients (Home Assistant).*

HTTP security headers (since v0.9.0)

All HTTP responses contain:

Header	Value
`X-Frame-Options`	`DENY`
`X-Content-Type-Options`	`nosniff`
`Referrer-Policy`	`same-origin`
`Content-Security-Policy`	`default-src 'self'; img-src 'self' data:; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline'; connect-src 'self'; frame-ancestors 'none'; base-uri 'self'; form-action 'self'`

`'unsafe-inline'` is required as long as templates use inline `onclick` handlers and `<script>` blocks. A later migration to `addEventListener` would replace this.

Resource caps

Aspect	Cap	Effect
Bus log	`MAX_BUSLOG_SIZE = 500` entries	Ring buffer, oldest entries drop out
SSE queue per client	`maxsize = 50` events	Slow clients are kicked out of the listener pool

Aspect	Cap	Effect
Total SSE listeners	`MAX_SSE_LISTENERS = 50` (since v0.9.0)	Anti-DoS: 51st `/api/v1/events` call gets `503`
JSON loader	`MAX_JSON_BYTES = 5 MB` (since v0.9.0)	Loader returns the default if `data/*.json` has been bloated

Secrets

- `DALI_SECRET_KEY` is generated automatically and stored in `data/secret_key` (mode 0600) if the ENV is unset. This keeps Flask sessions valid across container restarts.
- The `udev` rule restricts access to `/dev/hidraw*` to the `dali` group (mode 0660).

Health endpoint

`GET /healthz` (auth-free) is the dedicated endpoint for container health checks and reverse-proxy probes. It contains no device information, only `{"status":"ok","version":"..."}`.

16. Backup and Restore

Create a backup

```
# Linux installation
./Code/backup.sh /path/to/backup

# Docker
./Code/backup.sh /path/to/backup
```

The script backs up:

- The driver configuration (`driver_config.json`)
- All data in the data directory
- For Docker: the Docker volume `dali-servui_dali-data`

Restore a backup

```
# Linux installation
sudo ./Code/restore.sh /path/to/backup/dali-servui_backup_YYYYMMDD_HHMMSS.tar.gz

# Docker (specify destination directory)
./Code/restore.sh backup.tar.gz /path/to/docker-volume
```

The restore stops the service, copies the data and starts the service again automatically.

17. Troubleshooting

Common issues

Problem	Cause	Solution
“Not connected”	No USB adapter detected	check `lsusb`, check udev rule
Bus scan finds nothing	No bus voltage / no addresses	check DALI bus, run commissioning
Timeout on commands	Hardware communication disturbed	check USB cable, restart service
MikroE: no reaction	TX inversion wrong	DALI 2 Click: enable “TX inverted”
FTDI: connection failed	Kernel driver blocking	`sudo rmmod ftdi_sio usbserial`
GPIO: permission denied	Missing permission	add user to the `gpio` group

Log analysis

```
# Show live logs
sudo journalctl -u dali-servui -f

# Show errors only
sudo journalctl -u dali-servui -p err

# Enable debug mode
DALI_LOG_LEVEL=DEBUG sudo systemctl restart dali-servui
```

Dryrun mode

The test mode (`DALI_DRIVER=dryrun`) simulates all hardware operations. Queries are answered with `0xFF`. This mode is useful for:

- Testing the web interface without hardware
- Development and debugging
- Demonstration and training purposes

18. What's new in v1.0.0

v1.0.0 – 2026-05-07 (current version)

First public release. Functionally on the level of 0.9.0; the focus of this version: UI bug fixes, cleaning up the documentation and version consistency across all source files.

- **Bug fixed – device status always showed "green"**: The status column in `/devices` interpreted the internal `present` flag (bus reachability at the last scan) as "switched on". As a result, every device appeared as "active" even if it was off. The column now shows three states as a pill badge, analogous to the group members:
  - **On** (green) – `level > 0`
  - **Off** (grey) – `level == 0` and reachable on the bus
  - **Not reachable** (orange) – `present == False`
- **Improved visibility of the switching state**: The table row also gets a subtle indicator (left-hand colour bar plus a slight background), the same visual language as the group cards (`group-on/off/warning`).
- **Live update without reload**: The JS functions `deviceOn` , `deviceOff` and `deviceLevel` synchronise the badge, the row class and the slider value display directly after the API call (new helper `updateDeviceRow`).
- **Navbar "Kaffee?"** → **"Kaffee"** (DE), "Coffee?" → "Coffee" (EN).
- **Version consistency**: All 21 headers in Python, HTML, CSS, JS, Bash and Docker files have been unified to version 1.0.0 (deploy 2026-05-07). Previously, files carried versions between 0.1.0 and 0.6.0 depending on the time of the last change.
- **Documentation cleaned up**: Older documentation and review-report versions (v0.2.0, v0.8.0, v0.9.0) have been moved out of `Dokumentation/` and `Reviewbericht/` into `Versionssicherung/v<X.Y.Z>/` . Only the current version remains in the main folders.
- **README rework**: Feature list at version 1.0.0, hardware table with a status column (production vs. coming soon), full table of ENV variables, updated API overview.

v0.9.0 – 2026-05-05

- **HTTP security headers**: `X-Frame-Options` , `X-Content-Type-Options` , `Referrer-Policy` , `Content-Security-Policy`
- **SSE listener cap** `MAX_SSE_LISTENERS = 50` with a `503` reply on saturation
- **JSON loader hardening**: 5 MB size cap, typed allow-list for `load_driver_config`
- **Frontend auth**: token via `<meta name="x-api-token">` ; JS sends the `X-API-Token` header; `EventSource` uses the `?token=...` query
- **Buy-me-a-coffee link** in the navbar next to "Help"

v0.8.0 – 2026-05-05

- **MikroE coming-soon stamp** in the settings dialog on the driver cards
- **Healthz endpoint** `GET /healthz` (auth-free) for container health checks
- **SECRET_KEY auto-generation** persisted in `data/secret_key` (0600)
- **CORS origins via ENV** `DALI_CORS_ORIGINS` (comma list, default: empty)
- **Compose bug fixed**: duplicate `volumes:` block, USB mapping as default
- **Hardening**: label length cap, dashboard item schema validation, sniff duration floor
- **Thread-safe commissioning log operations**, signal handler in the app factory
- LICENSE file (full GPLv3 text) added

v0.7.0 – 2026-04-04

Multi-dashboard per room, REST API for dashboards, reusable group-card partials, default dashboard protected.

v0.6.0 – 2026-04-04

Group API with members and level, CORS header for the API, SSE stream `/api/v1/events` with keepalive.

v0.5.x – 2026-04-04

Commissioning with live stream (SSE), python-dali subprocess for Hasseb, CLI script `dali_commission.py`, bus sniffer, driver stop/start API, address reset and factory reset.

v0.4.x – 2026-04-03

Bus log with ring buffer and human-readable frame descriptions, group assignment via the UI (AddToGroup/RemoveFromGroup), persistent labels for devices and groups, Lucide icons in the navigation.

v0.3.x – 2026-03-30

DT6/DT8 (tunable white, RGB), feature toggles, extended API, bilingual help page with TOC, integrated reverse-proxy hint.

19. Appendix A – Checklist

Installation checklist

- ☐ Hardware connected and detected (`lsusb` or `gpioinfo`)

- ☐ Docker or Linux installation completed
- ☐ Web UI reachable at `http://<IP>:5000`
- ☐ Driver selected and activated in settings
- ☐ Bus scan run, devices detected
- ☐ Single-device control (on/off/level) works
- ☐ Group control works
- ☐ Broadcast control works

Security checklist

- ☐ `DALI_SECRET_KEY` is set **or** the persisted `data/secret_key` exists and is protected with 0600
- ☐ `DALI_API_TOKEN` is set (if the API is reachable over the LAN)
- ☐ `DALI_CORS_ORIGINS` is restricted to known frontend hosts (no `*`)
- ☐ Access to port 5000 is restricted by firewall
- ☐ HTTPS configured via a reverse proxy (e.g. Caddy, Traefik)
- ☐ Health check uses `/healthz` (auth-free) – container stays healthy

Hardware checklist

- ☐ DALI bus voltage measured (typ. 16V, min. 11.5V, max. 22.5V)
- ☐ Maximum bus load observed (250mA on Hasseb, external on MikroE)
- ☐ Maximum cable length observed (300m at 1.5mm^2, 100m at 0.5mm^2)
- ☐ All devices have short addresses (commissioning completed)